JavaScript is disabled on your browser.

Interface SortedMap<K,V>

Type Parameters:

`K - the type of keys maintained by this map`

`V - the type of mapped values`

All Superinterfaces:

`Map<K,V>`

All Known Subinterfaces:

`ConcurrentNavigableMap<K,V>, NavigableMap<K,V>`

All Known Implementing Classes:

`ConcurrentSkipListMap, TreeMap`

```
public interface SortedMap<K,V>
extends Map<K,V>
```

A `Map` that further provides a *total ordering* on its keys. The map is ordered according to the natural ordering of its keys, or by a `Comparator` typically provided at sorted map creation time. This order is reflected when iterating over the sorted map's collection views (returned by the `entrySet`, `keySet` and `values` methods). Several additional operations are provided to take advantage of the ordering. (This interface is the map analogue of `SortedSet`.)

All keys inserted into a sorted map must implement the `Comparable` interface (or be accepted by the specified comparator). Furthermore, all such keys must be *mutually comparable*: `k1.compareTo(k2)` (or `comparator.compare(k1, k2)`) must not throw a `ClassCastException` for any keys `k1` and `k2` in the sorted map. Attempts to violate this restriction will cause the offending method or constructor invocation to throw a `ClassCastException`.

Note that the ordering maintained by a sorted map (whether or not an explicit comparator is provided) must be *consistent with equals* if the sorted map is to

correctly implement the `Map` interface. (See the `Comparable` interface or `Comparator` interface for a precise definition of *consistent with equals*.) This is so because the `Map` interface is defined in terms of the `equals` operation, but a sorted map performs all key comparisons using its `compareTo` (or `compare`) method, so two keys that are deemed equal by this method are, from the standpoint of the sorted map, equal. The behavior of a tree map *is* well-defined even if its ordering is inconsistent with equals; it just fails to obey the general contract of the `Map` interface.

All general-purpose sorted map implementation classes should provide four "standard" constructors. It is not possible to enforce this recommendation though as required constructors cannot be specified by interfaces. The expected "standard" constructors for all sorted map implementations are:

1. A void (no arguments) constructor, which creates an empty sorted map sorted according to the natural ordering of its keys.
2. A constructor with a single argument of type `Comparator`, which creates an empty sorted map sorted according to the specified comparator.
3. A constructor with a single argument of type `Map`, which creates a new map with the same key-value mappings as its argument, sorted according to the keys' natural ordering.
4. A constructor with a single argument of type `SortedMap`, which creates a new sorted map with the same key-value mappings and the same ordering as the input sorted map.

Note: several methods return submaps with restricted key ranges. Such ranges are *half-open*, that is, they include their low endpoint but not their high endpoint (where applicable). If you need a *closed range* (which includes both endpoints), and the key type allows for calculation of the successor of a given key, merely request the subrange from `lowEndpoint` to `successor(highEndpoint)`. For example, suppose that `m` is a map whose keys are strings. The following idiom obtains a view containing all of the key-value mappings in `m` whose keys are between `low` and `high`, inclusive:

```
SortedMap<String, V> sub = m.subMap(low, high+"\0");
```

A similar technique can be used to generate an *open range* (which contains neither endpoint). The following idiom obtains a view containing all of the key-value mappings in `m` whose keys are between `low` and `high`, exclusive:

```
SortedMap<String, V> sub = m.subMap(low+"\0", high);
```

This interface is a member of the Java Collections Framework.

Since:

`1.2`

See Also:

`Map`, `TreeMap`, `SortedSet`, `Comparator`, `Comparable`, `Collection`, `ClassCastException`

Nested Class Summary

Nested classes/interfaces inherited from interface java.util.Map

`Map.Entry<K,V>`

Method Summary

Abstract Methods

Modifier and Type	Method	Description
`Comparator<? super K>`	`comparator()`	Returns the comparator used to order the keys in this map, or `null` if this map uses the **natural ordering** of its keys.
`Set<Map.Entry<K,V>>`	`entrySet()`	Returns a `Set` view of the mappings contained in this map.
`K`	`firstKey()`	Returns the first (lowest) key currently in this map.
`SortedMap<K,V>`	`headMap(K toKey)`	Returns a view of the portion of this map whose keys are strictly less than `toKey`.
`Set<K>`	`keySet()`	Returns a `Set` view of the keys contained in this map.
`K`	`lastKey()`	Returns the last (highest) key currently in this map.
`SortedMap<K,V>`	`subMap(K fromKey, K toKey)`	Returns a view of the portion of this map whose keys range from `fromKey`, inclusive, to `toKey`, exclusive.
`SortedMap<K,V>`	`tailMap(K fromKey)`	Returns a view of the portion of this map whose

		keys are greater than or equal to `fromKey`.
`Collection<V>`	**`values`**`()`	Returns a **`Collection`** view of the values contained in this map.

Methods inherited from interface java.util.Map

`clear`, `compute`, `computeIfAbsent`, `computeIfPresent`, `containsKey`, `containsValue`, `equals`, `forEach`, `get`, `getOrDefault`, `hashCode`, `isEmpty`, `merge`, `put`, `putAll`, `putIfAbsent`, `remove`, `remove`, `replace`, `replace`, `replaceAll`, `size`

Method Detail

comparator

```
Comparator<? super K> comparator()
```

Returns the comparator used to order the keys in this map, or `null` if this map uses the natural ordering of its keys.

Returns:

`the comparator used to order the keys in this map, or null if this map uses the natural ordering of its keys`

subMap

```
SortedMap<K,V> subMap(K fromKey,
                      K toKey)
```

Returns a view of the portion of this map whose keys range from `fromKey`, inclusive, to `toKey`, exclusive. (If `fromKey` and `toKey` are equal, the returned map is empty.) The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside its range.

Parameters:

`fromKey - low endpoint (inclusive) of the keys in the returned map`

`toKey - high endpoint (exclusive) of the keys in the returned map`

Returns:

a view of the portion of this map whose keys range from fromKey, inclusive, to toKey, exclusive

Throws:

ClassCastException - if fromKey and toKey cannot be compared to one another using this map's comparator (or, if the map has no comparator, using natural ordering). Implementations may, but are not required to, throw this exception if fromKey or toKey cannot be compared to keys currently in the map.

NullPointerException - if fromKey or toKey is null and this map does not permit null keys

IllegalArgumentException - if fromKey is greater than toKey; or if this map itself has a restricted range, and fromKey or toKey lies outside the bounds of the range

headMap

SortedMap<K,V> headMap(K toKey)

Returns a view of the portion of this map whose keys are strictly less than toKey. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an IllegalArgumentException on an attempt to insert a key outside its range.

Parameters:

toKey - high endpoint (exclusive) of the keys in the returned map

Returns:

a view of the portion of this map whose keys are strictly less than toKey

Throws:

ClassCastException - if toKey is not compatible with this map's comparator (or, if the map has no comparator, if toKey does not implement Comparable). Implementations may, but are not required to, throw this exception if toKey cannot be compared to keys currently in the map.

NullPointerException - if toKey is null and this map does not permit null keys

IllegalArgumentException - if this map itself has a restricted range, and toKey lies outside the bounds of the range

tailMap

`SortedMap<K,V> tailMap(K fromKey)`

Returns a view of the portion of this map whose keys are greater than or equal to `fromKey`. The returned map is backed by this map, so changes in the returned map are reflected in this map, and vice-versa. The returned map supports all optional map operations that this map supports.

The returned map will throw an `IllegalArgumentException` on an attempt to insert a key outside its range.

Parameters:

`fromKey - low endpoint (inclusive) of the keys in the returned map`

Returns:

`a view of the portion of this map whose keys are greater than or equal to fromKey`

Throws:

`ClassCastException - if fromKey is not compatible with this map's comparator (or, if the map has no comparator, if fromKey does not implement Comparable). Implementations may, but are not required to, throw this exception if fromKey cannot be compared to keys currently in the map.`

`NullPointerException - if fromKey is null and this map does not permit null keys`

`IllegalArgumentException - if this map itself has a restricted range, and fromKey lies outside the bounds of the range`

firstKey

`K firstKey()`

Returns the first (lowest) key currently in this map.

Returns:

`the first (lowest) key currently in this map`

Throws:

`NoSuchElementException - if this map is empty`

lastKey

`K lastKey()`

Returns the last (highest) key currently in this map.

Returns:

the last (highest) key currently in this map

Throws:

`NoSuchElementException` - if this map is empty

keySet

`Set<K> keySet()`

Returns a `Set` view of the keys contained in this map. The set's iterator returns the keys in ascending order. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. If the map is modified while an iteration over the set is in progress (except through the iterator's own `remove` operation), the results of the iteration are undefined. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll`, and `clear` operations. It does not support the `add` or `addAll` operations.

Specified by:

`keySet` in interface `Map<K,V>`

Returns:

a set view of the keys contained in this map, sorted in ascending order

values

`Collection<V> values()`

Returns a `Collection` view of the values contained in this map. The collection's iterator returns the values in ascending order of the corresponding keys. The collection is backed by the map, so changes to the map are reflected in the collection, and vice-versa. If the map is modified while an iteration over the collection is in progress (except through the iterator's own `remove` operation), the results of the iteration are undefined. The collection supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Collection.remove`, `removeAll`, `retainAll` and `clear` operations. It does not support the `add` or `addAll` operations.

Specified by:

`values in interface Map<K,V>`

Returns:

`a collection view of the values contained in this map, sorted in ascending key order`

entrySet

`Set<Map.Entry<K,V>> entrySet()`

Returns a `Set` view of the mappings contained in this map. The set's iterator returns the entries in ascending key order. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. If the map is modified while an iteration over the set is in progress (except through the iterator's own `remove` operation, or through the `setValue` operation on a map entry returned by the iterator) the results of the iteration are undefined. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll` and `clear` operations. It does not support the `add` or `addAll` operations.

Specified by:

`entrySet in interface Map<K,V>`

Returns:

C

```
a set view of the mappings contained in this map, sorted in
ascending key order
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.